

Exercise 3: Sorting Customer Orders

Scenario

You are tasked with sorting customer orders by their total price on an e-commerce platform. This helps in prioritizing high-value orders.

1. Understand Sorting Algorithms

Bubble Sort

- Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order.
- It is simple to implement but inefficient for large datasets.
- Time Complexity:
 - Best Case: $O(n)$ (Already Sorted)
 - Average Case: $O(n^2)$
 - Worst Case: $O(n^2)$

Insertion Sort

- Insertion Sort builds the sorted array one element at a time by inserting each element into its correct position.
- Efficient for small or nearly sorted datasets.
- Time Complexity:
 - Best Case: $O(n)$
 - Average Case: $O(n^2)$
 - Worst Case: $O(n^2)$

Quick Sort

- Quick Sort uses the Divide and Conquer technique.
- It selects a pivot element, partitions the array into smaller subarrays, and recursively sorts them.
- Time Complexity:
 - Best Case: $O(n \log n)$
 - Average Case: $O(n \log n)$
 - Worst Case: $O(n^2)$

Merge Sort

- Merge Sort divides the array into two halves, recursively sorts each half, and merges them.
- Provides consistent performance regardless of input.
- Time Complexity:
 - Best Case: $O(n \log n)$
 - Average Case: $O(n \log n)$

- Worst Case: $O(n \log n)$

2. Setup

A Java project named `SortingCustomerOrders` was created.

The project contains a code folder with the following files:

`SortingCustomerOrders`

└── code

 ├── Order.java

 └── SortingCustomerOrders.java

- `Order.java` – Defines the `Order` class with `orderId`, `customerName`, and `totalPrice`.
- `SortingCustomerOrders.java` – Implements Bubble Sort and Quick Sort and demonstrates sorting customer orders.

3. Implementation

Order Class

The `Order` class contains the following attributes:

- `orderId`
- `customerName`
- `totalPrice`

Sorting Algorithms Implemented

Bubble Sort

- Compares adjacent orders based on `totalPrice`.
- Swaps them whenever they are in the wrong order.
- Continues until the entire array is sorted.

Quick Sort

- Selects a pivot element.
- Partitions the array around the pivot.
- Recursively sorts the left and right partitions.
- Efficient for large datasets.

4. Analysis

Time Complexity Comparison

Algorithm	Best Case	Average Case	Worst Case
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$

Why Quick Sort is Preferred

- Bubble Sort performs multiple unnecessary comparisons and swaps, making it inefficient for large datasets.
- Quick Sort reduces the number of comparisons by partitioning the array into smaller subarrays.
- On average, Quick Sort performs significantly faster than Bubble Sort.
- Therefore, Quick Sort is the preferred choice for sorting large collections of customer orders in an e-commerce platform.

5. Conclusion

The customer orders were successfully sorted using both Bubble Sort and Quick Sort. By comparing their time complexities, it is evident that Quick Sort provides much better performance for large datasets. Hence, Quick Sort is generally preferred in real-world e-commerce applications where efficient sorting is essential.